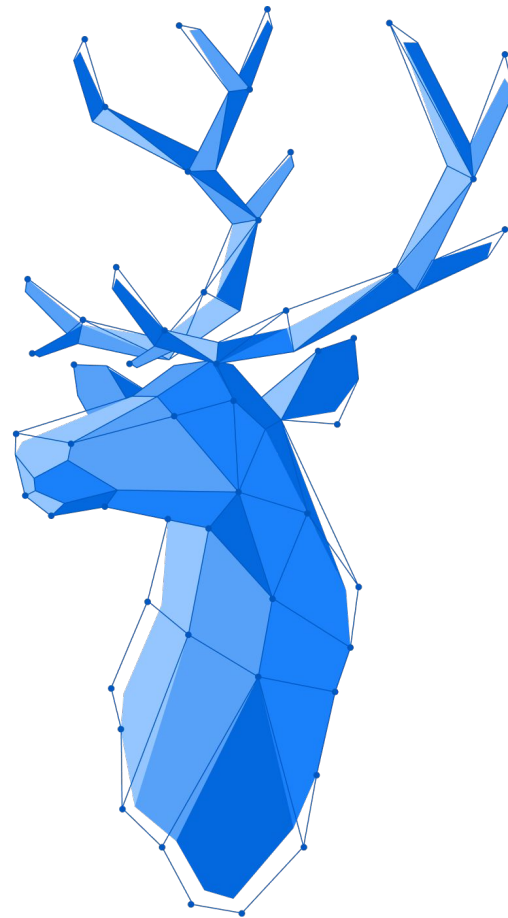




Asynchrone Prozesse

Von Gerald Koch



Vorwort/Agenda

- Alle UI Frameworks (Spring, Vaadin, JavaFX, etc.) haben etwas gemeinsam: sie frieren bei lang laufenden Prozesse, wie z.B. Exports ein. Um die UI wieder "aufzutauen", sollten diese in einem anderen Thread ausgeführt werden. Um von einem Thread wieder in den UI Thread zu kommen, gibt es spezielle Methoden:
 - **Swing:** `Swing.invokeLater(Runnable runnable)`
 - **Vaadin:** `UI.access(Command command)`
- Zusätzlich dazu haben wir ein Problem wenn wir Spring benutzen wenn wir den `SecurityContext` nicht in die anderen Threads kopieren.

Wie wir diese Themen angehen, wird in den folgenden Kapiteln erklärt:

- Schritt 1: Lang laufende Prozesse in einem extra Thread ausführen
- Schritt 2: Den Spring Context mitnehmen
- Schritt 3: Einbeziehung der Vaadin UI
- Schritt 4: Progress Dialog

Asynchrone Prozesse

Schritt 1 – Prozess in extra Thread

Eine `CompletableFuture` ist ein guter weg, um Aufgaben in einem Thread auszuführen.

Folgendes ist zu beachten:

Falls ein Rückgabewert weiterverarbeitet werden soll, benutzt man folgende Kette:

```
CompletableFuture
```

```
    .supplyAsync(Supplier<U> longRunningTask)  
    .whenComplete(BiConsumer<? super T, ? super Throwable> processResult)
```

Wenn es keinen Rückgabewert gibt, kann man folgende Kette verwenden:

```
CompletableFuture
```

```
    .runAsync(Runnable longRunningTask)  
    .whenComplete(BiConsumer<Void, ? super Throwable> executeAfter)
```

Achtung: `supplyAsync/runAsync` und `whenComplete` benutzen jeweils einen anderen Thread! Threads haben nicht den gleichen Spring Kontext wie der UI Thread.

Schritt 2 – Spring Kontext in Threads

Der UI Thread einer Spring application besitzt einen Security Kontext mit Informationen über den authentifizierten Benutzer. Der Spring Kontext für nicht-UI Threads sind nicht gesetzt, Also werden Aufrufe wie

`SecurityContextHolder.getContext().getAuthentication().getPrincipal()`
null zurückliefern oder eine Exception hervorbringen.

Diese Information muss dem ausführenden Thread unter Benutzung der `SecurityContextHolder` Methoden `getContext()` und `setContext(SecurityContext context)` übergeben werden. Bitte denken Sie auch daran, den Kontext nach Verwendung des Threads zurückzusetzen. Idealerweise sollten Sie die Aufgabe eines Threads in einem try-finally Konstrukt ausführen und im finally Block den Reset durchführen, um sicher zu stellen, dass es ausgeführt wird.

Schritt 2 – Abstraktes Code Beispiel

```
// save context of UI thread
SecurityContext uiContext = SecurityContextHolder.getContext();
// start new thread
CompletableFuture.runAsync(() -> {
    // save thread context
    SecurityContext context = SecurityContextHolder.getContext();
    // set context of the UI thread
    SecurityContextHolder.setContext(uiContext);
    try {
        // execute thread stuff
    }
    finally {
        // restore thread context
        SecurityContextHolder.setContext(context);
    } // end thread
}).whenComplete((result, error) -> { ... });
```

Schritt 2 – Optimierungen

```
public static void doInContext(SecurityContext outerContext, Runnable runnable){
    SecurityContext threadContext = SecurityContextHolder.getContext();
    SecurityContextHolder.setContext(outerContext);
    try { runnable.run(); } // execute runnable
    finally { SecurityContextHolder.setContext(threadContext); }
}

public static <T> T doInContext(SecurityContext outerContext, Supplier<T> supplier){
    SecurityContext threadContext = SecurityContextHolder.getContext();
    SecurityContextHolder.setContext(outerContext);
    try { return supplier.get(); } // execute supplier
    finally { SecurityContextHolder.setContext(threadContext); }
}
```

Schritt 3 – Zurück zur UI

Wenn asynchrone Threads ausgeführt werden, können wir nicht einfach UI Methoden aufrufen. Solche UI Aktionen wie, Erfolgsmeldungen anzeigen und Fortschrittsbalken updaten, muss im UI Thread getan werden. Für den Fall dass UI Funktionen von nicht-UI Threads ausgeführt werden sollen, gibt es spezielle Methoden um auf den UI Thread zuzugreifen:

- Swing: statischer Aufruf von `Swing.invokeLater(...)`
- Vaadin: nicht statischer Aufruf von `UI.access(...)`, denn Vaadin ist eine multi-user Umgebung. Es gibt demnach auch mehrere UIs. Diese Methode muss also auf der Instanz einer UI Klasse aufgerufen werden.

Um es abschließend zu ermöglichen, dass Vaadin ein update der UI anstößt, muss "Push" eingeschaltet werden. Dafür annotieren wir unsere Klasse `Application` mit `@Push`.

Schritt 3 – Code Beispiel

```
class UiDemo extends FlexLayout {  
    private final Button button = new Button("Demo", e -> doMultiThreaded());  
    public UiDemo() {  
        button.setDisableOnClick(true);  
        add(button);  
    }  
    public void doMultiThreaded() {  
        SecurityContext uiContext = SecurityContextHolder.getContext(); // save context of UI thread  
        Optional<UI> ui = getUi(); //fetch the UI object  
        CompletableFuture // setup completable future structure (step 1)  
            .supplyAsync(() -> SecurityUtils.doInContext(uiContext, () -> longRunning(...)))  
            .whenComplete((result, error) ->  
                ui.ifPresent(u -> { //run when UI is attached  
                    if(u.isClosing()) { return; } //check if UI is closing or was closed  
                    // call the UI.access method and set the button enabled again  
                    u.access(() -> button.setEnabled(true));  
                }));  
    }  
}
```


Schritt 3 – Optimierungen

```
public static void doInUiThread(Component component, Consumer<UI> command) {  
    doInUiThread(component.getUI(), command);  
}  
  
public static void doInUiThread(Optional<UI> ui, Consumer<UI> command) {  
    ui.ifPresent(u -> doInUiThread(u, command));  
}  
  
public static void doInUiThread(UI ui, Consumer<UI> command) {  
    if(ui.isClosing()) {  
        return;  
    }  
    ui.access(() -> command.accept(ui));  
}
```

Schritt 4 – Progress Dialog

- Wir haben gelernt:
 - Threads mit `CompletableFuture` zu erzeugen
 - Den Spring Kontext in anderen Threads zu verwenden
 - Auf die UI von nicht-UI threads zuzugreifen
- Progress Dialog:
 - Mehrere Updates der UI während ein lang laufender Prozess ausgeführt wird
 - `ProgressListener` für die Kommunikation zwischen Prozess und UI
 - Schließt wenn lang laufender Prozess endet oder fehlschlägt
 - Sollte idealerweise den aktuellen Fortschritt relativ zum Ziel anzeigen
 - **Achtung:** Zu viele UI Updates können in schlechter Performance resultieren

Schritt 4 – Progress Listener

Das ProgressListener Interface wird verwendet, um das maximum und den aktuellen Wert zu erhalten.

```
@FunctionalInterface
public interface ProgressListener {
    void setProgress(double max, double current);
}
```

Schritt 4 – Progress Dialog Eigenschaften

- Implementiert ProgressListener
- Greift automatisch auf den UI Thread beim aufruf von setProgress(...) zu
- Berechnet automatisch das ETA nach 10s
- Zusätzliche Methoden
 - Konstruktor mit Title
 - Attribute im Builderstyle
 - Title
 - Unit

Schritt 4 – Beispielprozess

```
class Snail {
    // create a collection to save all the listeners
    private Collection<ProgressListener> listeners = Collections.synchronizedSet(new    LinkedHashSet<>());
    public void creep (int iterations, long sleepTime) {
        for(int i = 0; i < iterations; i++) {
            try {
                Thread.sleep(sleepTime);
                fireProgress(iterations, i + 1); // fire listener
            } catch (Exception ignored) {}
        }
    }
    private void fireProgress(double max, double current) { // iterate over the listeners and fire the values
        new LinkedHashSet<>(listeners).forEach(l -> l.setProgress(max, current));
    }
    public Registration addProgressListener(ProgressListener listener) { // register listener
        // Registration is a class which allows to keep a listener object and remove it to avoid memory leaks
        return Registration.addAndRemove(listeners, listener);
    }
}
```

Schritt 4 – Beispiel auf UI Seite

```
class UiDemo extends FlexLayout {  
    private final Button button = new Button("Demo", e -> doLongRunningTask());  
    public UiDemo() {  
        button.setDisableOnClick(true);  
        add(button);  
    }  
    public void doSomethingInTheUiFromAnotherThread() {  
        ProgressDialog pDialog = new ProgressDialog("Traveling the World").withUnit(" mm").open();  
        Snail snail = new Snail();  
        snail.addProgressListener(pDialog);  
        SecurityContext uiCtxt = SecurityContextHolder.getContext(); // save UI context  
        CompletableFuture.supplyAsync(() -> doInContext(uiCtxt, snail.creep(200, 100))  
            .whenComplete((result, error) -> doInUiThread(this, () -> {  
                button.setEnabled(true);  
                pDialog.close();  
            })));  
    }  
}
```

Schritt 4 – Endergebnis

Traveling the World

ETA: 06s

450/500 mm

